Qubit & Toffoli Reduction Techniques

A Comprehensive Primer

ecdsa.fail quantum circuit benchmark

Based on research notes from the trailmix_ludicrous optimization track

Audience note

An undergrad who has learned the basics of quantum computing — superposition, entanglement, Toffoli/CNOT/Hadamard gates, quantum circuits, and reversible computation. This document explains every optimization technique used in the ecdsa.fail challenge from circuit-level first principles, with concrete examples from actual research.

As of 2026-06-27

Contents

1. What We Are Optimizing

1.A. The score

Score objective (lower is better)

$$\text{score} = \text{peak\_qubits} \times \text{avg\_executed\_Toffoli}$$

This is the ecdsa.fail benchmark's cost function for a quantum circuit that computes an affine elliptic-curve point addition on secp256k1 (Bitcoin's curve). The two factors penalize two different resources:

- **Peak qubits**: the maximum number of qubits simultaneously in use at any instant. This is the "width" of the circuit, not the total ever allocated. On real hardware, idle qubits still occupy physical space on the quantum processor.
- **Average executed Toffoli**: the Toffoli (CCX) gate count, *weighted by how often each gate fires*, averaged over the 9024 test inputs the challenge uses. Not just the gate count, but gates × firing_rate.

1.B. Why Toffoli gates specifically?

On a **fault-tolerant** quantum computer (the kind needed to break Bitcoin-scale cryptography), gates fall into two classes:

- **Clifford gates** (CNOT, Hadamard H, Phase S, Pauli $X/Y/Z$): implemented efficiently using the surface error-correcting code at very low physical-qubit overhead.
- **Non-Clifford gates** (Toffoli/ T-gate): require **magic states** — specially prepared ancilla qubits that take a long time and many physical qubits to distill via "magic state distillation." Each Toffoli gate consumes one distilled magic state.

The rough conversion: 1 Toffoli = exactly 7 T-gates (Gosset et al. 2013, tight lower bound).

So in this regime, **Toffoli gates are like currency**: each one costs approximately the same fixed price (one magic state). The peak qubit count is like "desk space" — idle qubits still occupy factory floor. The product peak_qubits × Toffoli captures the spacetime volume of running the circuit.

Common metric confusion

This challenge uses `avg_executed_Toffoli`, NOT:

- T-count (= 7× Toffoli, a different metric)
- T-depth (Selinger 2013 — a depth trick with 4 ancilla per Toffoli; irrelevant here)

The distinction matters: average-executed rewards conditional execution (replay), emitted T-count does not.

1.C. The current SOTA

The best known result as of 2026-06-27 (BitWonka, commit `d44cad3`, trailmix_ludicrous family):

$$1152 \text{ qubits} \times 1{,}364{,}230 \text{ avg Toffoli} = 1{,}571{,}592{,}960 \text{ score}$$

1.D. Two separate competitions, two separate objectives

This is important context for the whole document. There are **two distinct optimization tracks**, and a technique that is optimal for one can be irrelevant for the other:

1. **The Q×T score competition** (the main subject of this primer): minimize the *product* `peak_qubits` × `avg_executed_Toffoli`. Here both resources matter, and every move is judged by the exchange rate (§9). The SOTA above (1152q × 1.36M) lives in this track.
2. **The low-qubit competition** (a separate objective): minimize **peak qubits alone**. Toffoli count is *unconstrained* — you may spend as many Toffoli gates as you like. The only thing that counts is how few qubits the circuit is simultaneously live on.

These are genuinely different games. Under the Q×T objective you would never spend 8× more Toffoli to save 10% of qubits; under the low-qubit objective that trade is *free*, because Toffoli simply does not appear in the scoring. Most of this primer optimizes track 1, but the deepest qubit-floor technique — **EEA register sharing (§6.O)** — belongs to track 2, and reaches qubit counts (851q → 829q) far below anything competitive on the product score. Keep the two objectives separate in your head: many "is this worth it?" questions only have an answer once you know *which* competition you are playing.

2. The Reversibility Constraint

2.A. You cannot delete information

Quantum circuits must be reversible. This is not a convention — it is required by quantum mechanics. Every gate in a quantum circuit must have an inverse (you can always "run it backwards"). This means:

- **You cannot overwrite a qubit.** If qubit A holds value x and you want to compute $f(x)$ into A, you cannot just replace it. You'd lose x forever, breaking reversibility.
- **You cannot erase a qubit.** If a qubit is entangled with the rest of the system (as intermediate results always are), setting it to $|0\rangle$ forcibly would violate unitarity.

The only way to "free" a qubit is to **uncompute** it — run the computation that created its value in reverse, returning it exactly to $|0\rangle$. Once it's $|0\rangle$, it can be freed.

2.B. Consequence: intermediate values cost qubits until uncomputed

Every time your circuit computes an intermediate result (a carry bit, a partial product, a comparison flag), it must allocate a fresh qubit. That qubit remains **live** — occupying a slot in the peak count — from the moment it's computed until the moment it's uncomputed.

Example: An n-bit addition $a + b$ computes n carry bits, one per bit position. Each carry qubit is live from when it's computed until the carry chain is reversed. If you compute the addition at step 100 and uncompute it at step 200, those n carry qubits are live across the entire steps 100–200, contributing to the peak at any instant in that range.

This is why almost every qubit-reduction technique boils down to: **make intermediate values die sooner** (so they aren't live at the peak) **or don't materialize them at all**.

2.C. Reversibility also doubles Toffoli counts

In a classical circuit, you compute X, use X, then discard X. In a reversible circuit, you compute X (costs Toffoli), use X, then *uncompute* X (costs the same Toffoli again). So every logical AND operation normally costs 2× — once forward, once backward.

This is the central pain point that Measurement-Based Uncomputation (§8.A) solves.

3. The Challenge Circuit

3.A. What it computes

The circuit computes one **affine elliptic curve point addition**: given quantum register $P = (P_x, P_y)$ and classical constant $Q = (Q_x, Q_y)$, compute $P + Q = R$ in place, where the arithmetic is modulo the secp256k1 prime $p = 2^{256} - 2^{32} - 977$.

The mathematical operations, in order:

```
dx = Px - Qx              (mod p subtraction)
dy = Py - Qy
lambda = dy / dx          (modular inverse + multiply: the expensive GCD step)
new_Px = lambda^2 - Px - Qx
new_Py = lambda*(Px - new_Px) - Py
```

3.B. Why modular inverse is the bottleneck

Computing $dy/dx \bmod p$ requires computing $dx^{-1} \bmod p$, the modular inverse of dx. There is no simple formula — it requires running the **Extended Euclidean Algorithm (EEA)** or equivalent, which takes ≈ 530 steps of binary GCD (divstep) operations. Each step involves:

- A comparison (swap decision: is $|u| > |v|$?)
- A conditional swap of two 256-bit registers
- A conditional subtraction of two 256-bit registers

This accounts for roughly 90% of the circuit's Toffoli count.

3.C. The key design choice: Q is classical

The second point Q is a **fixed constant** (the secp256k1 generator point, known at circuit compile time). The current SOTA keeps Q as classical bit-patterns (`BitIds`) rather than quantum registers, eliminating **512 qubits** from the peak — the single largest qubit reduction in the circuit's history.

4. Measuring Progress

4.A. The live-qubit-vs-time profile

To reduce the peak, you first need to know **where** the peak occurs. Plot how many qubits are live at each moment in the circuit timeline:

```
qubits  |
live    |         ##########
        |    ####################
        |  #########################
        |################################
        +--+----+--+------+--+----+----> time
                   ^
                   peak = 1152 here (the "binder")
```

The highest point is the **peak**, and the phase executing at that height is the **binder**. Only operations alive at the binder instant matter for qubit reduction.

Key tool: `TRACE_PEAK=1 TRACE_PHASE_ACTIVE=1` in the build flags. This instruments every `alloc_qubit`/`free_qubit` call and prints the name of the phase executing when the maximum is hit.

4.B. The plateau problem

After shaving the single tallest binder, you often discover the peak hasn't moved — because several *independent* phases all reach the same height. This is a **co-binder plateau**:

```
qubits  |
live    |    #####   #####     ####
        |  #########################
        |  #########################  <- plateau at 1152 (Phase A, B, C all tie here)
        +----+--+----+---+--+---+---> time
```

The reported peak doesn't drop until **every** plateau member is individually pushed below the current height. This changes the strategy fundamentally: you must work in "plateau mode" — find a local qubit reduction for each co-binder, then combine them all.

4.C. Emitted vs average-executed Toffoli

- **Emitted**: the raw count of CCX gates in the circuit, regardless of whether they fire.
- **Average-executed**: each CCX gate's contribution weighted by Pr[control fires], averaged over all 9024 test inputs. A CCX that only fires 5% of the time contributes only 0.05.

This distinction is critical because **conditional replay** (§8.C) can dramatically reduce average-executed without changing emitted at all.

5. Qubit Reduction: The Core Loop

Before diving into individual techniques, understand the general loop:

1. **Measure** — get the live-qubit-vs-time trace; find the peak height and the binder phase.
2. **Inventory** — list every qubit alive at the binder instant.
3. **Classify** — label each qubit by category (table below).
4. **Brainstorm** — for each non-essential qubit at the binder, propose a shave.
5. **Cost** — compute the Toffoli overhead vs qubits saved. Compare to the break-even exchange rate (§9).
6. **Re-measure** — apply the cheapest safe shave. The peak moves to a new binder. Repeat.

5.A. Qubit classification at the binder

Class	What it holds	Shrinkable?
Essential data	Inputs/outputs the binder reads or writes	Rarely
Scratch	Temporary arithmetic results (carries, partial products)	Often
Transcript / log	GCD branch decisions recorded for reversibility	Often — compress the log
Passenger	Allocated before the peak, consumed after, *idle at binder*	Yes — relocate its endpoints
Alias-candidate	Restorable duplicate of a value already live elsewhere	Yes — borrow as dirty scratch
Truncatable	Bits provably zero or irrelevant to the final output	Yes

5.B. Binary vs graded failure modes

A critical distinction before applying any technique:

- **Graded-failure levers** (carry truncation, comparator narrowing): correctness degrades *continuously* with aggressiveness. A small fraction of inputs go wrong. You can iterate on the parameter and search for a nonce where all 9024 test inputs avoid the failures.
- **Binary-failure levers** (register live-range holes, transcript compression, structural reuse): correctness is all-or-nothing. Either the circuit is value-exact on all inputs, or it's wrong on essentially *all* inputs. There is no graded near-miss ladder to climb. You must get the implementation mathematically correct before searching for nonces.

The trap

Applying the "tweak-and-triage" loop to a binary-failure lever. A structural width cut that's not value-exact will produce all-dirty garbage on every input, not near-misses.

6. Qubit Reduction Techniques

6.A. Live-Range Holes: Uncompute Early, Recompute Later

The headline move. If a qubit holds a value V that the peak instant does NOT need (it was computed before the peak and will be consumed after), free it before the peak and rebuild it after.

```
Timeline view:
Before:  ...[V computed]=====[V held live across peak]=====[V consumed]...
After:   ...[V computed][uncompute V]  (peak happens)  [recompute V][V consumed]...
                                        ^^^^^^^^^^^
                                   V is absent at peak
```

Mechanics: Emit the reverse of the gates that built V (returning its qubits to $|0\rangle$ and freeing them), run the peak-owning phase with that freed headroom, then re-emit the forward gates to restore V for its later consumer.

Cost: For a value that cost C_0 Toffoli to compute:

- Save k qubits at the peak
- Spend $2C_0$ extra Toffoli (one uncompute + one recompute)
- The trade wins if $2C_0 < k \times$ (avgT/peak_q) (the break-even rate)

Real examples:

- **1211→1193**: A GCD-apply scratch block was uncomputed during a shift sub-phase that didn't use it, then reacquired after. Saved 18 qubits.
- **1166→1165** (`TLM_PARK_ODD_U0`): `u[0]` is provably 1 (GCD odd-u invariant). Apply $X \to |0\rangle$ to zero it, `loan_zero_qubit` to return the lane, then `restore_known_one` (apply X again) to restore. Near-zero Toffoli cost, -1 qubit.

The keep-alive dual: Sometimes the uncompute+recompute round-trip creates a wider transient intermediate than just holding the value. In that case, *keep* the value live across the middle operation. Choose whichever is narrower: holding footprint vs recompute peak transient.

6.B. Passenger Relocation

A **passenger** is a qubit that was allocated early and consumed late, but does nothing at the peak — it just sits idle.

The cheaper fix: Don't uncompute and recompute. Just **allocate later** (right before first use) and **free earlier** (right after last use). If both endpoints fall outside the peak window, the passenger vanishes from the binder count at zero Toffoli cost.

Check passengers first before resorting to live-range holes — relocation is free.

6.C. Range-Bound a Register to Drop a Guard Bit

Some registers need an extra "guard bit" (sign or overflow indicator) only because an intermediate could transiently go negative or overflow. If you can prove the value always stays in a non-negative range by reordering operations, delete the guard bit entirely.

Example (benhuang Lever B): The fold quotient register had bit 33 (a sign guard) because Solinas reduction terms could make the partial sum dip below zero. Reorder to apply positives first. Now the sum is always in $[0, 2^{33})$. Allocate 33 bits instead of 34. **Net:** -1 **qubit at zero Toffoli cost.** This produced the 1221→1220 drop.

General rule: In $+A - B$ sequences, applying positives first and subtracting last keeps partial sums non-negative, eliminating sign-guard bits.

6.D. Truncation: Keep Only the Bits That Matter

Carry chains, high words of partial sums, and intermediate registers often hold more bits than actually affect the final result.

Per-step scheduling: In a 530-step loop (the GCD), a single constant truncation width must be set safe for the worst iteration. A per-step schedule width(step) lets each iteration be exactly as tight as it can tolerate — saving qubits only at the steps that are actually at the peak.

Two truncation flavors:

- **Structural truncation (density-neutral)**: prove from circuit analysis that the dropped bits are always zero on all inputs.

- **Empirical truncation (island-exact)**: verify the dropped bits are never 1 on the 9024 challenge inputs specifically. Requires re-hunting a nonce after each change.

6.E. Free-and-Recompute a Cheap Value (The 1153→1152 Breakthrough)

When a dead-qubit scan finds **no** unused qubits at the peak, don't conclude "the floor is structural." Ask instead:

> *Is any held value a cheap function of the other values already live at the peak?*

If yes, **free** that qubit, let the peak phase use its slot, then **recompute** the value from the live qubits afterward for negligible cost.

The cy0 trick (`d44cad3`, BitWonka): The qubit `cy0` holds the carry-in at bit 0. No dead qubits found by scan. But because the secp256k1 prime constant $f[0] = 1$:

$$\texttt{cy0} = \texttt{control\_bit}\ \text{AND}\ (\text{NOT}\ \texttt{final\_a0})$$

Both `control_bit` and `final_a0` are already live at the peak. So:

1. Uncompute `cy0` using 2 CNOT-equivalent gates (≈ 0 Toffoli)
2. Return the freed lane via `loan_zero_qubit`
3. After the peak phase: recompute `cy0` from `control_bit` and `final_a0` (2 gates)

Applied 40 times (once per binding fold iteration): 40×4 CNOTs $= 0$ Toffoli. Saves 1 qubit. The 1153→1152 drop.

Generalized lesson: After a dead-qubit scan fails, scan again for "whose value is a CHEAP FUNCTION of other live qubits?" The second question catches what the first misses.

6.F. Dynamic-Width Registers

In a GCD algorithm, operands naturally shrink as computation proceeds. After k steps of binary GCD, the operands u and v have lost at most k bits from their high ends. A naive circuit allocates full 256-bit registers for all 530 steps.

Dynamic-width registers: At each step i, determine the actual live bit-width and only allocate that many qubits. Free the high bits as they become known-zero.

Ablation results (from the Shrunken-PZ q980 track):

```
Source-level dynamic PZ (universal schedule):     1027 qubits
+ thin support-tuned schedule (per-step widths):   990 qubits  (-37q)
+ counter/rotation metadata trimming:              983 qubits   (-7q)
+ quotient cap:                                    980 qubits   (-3q)
```

Thin schedule (island-exact): An even tighter version samples actual GCD widths over many random inputs and builds per-step bounds tight to that distribution. This is island-exact — the thin schedule may be too narrow for inputs outside the sampled set. Requires nonce hunting.

6.G. Known-Constant Teardown Before the Peak

When the GCD terminates, certain registers are in known, predetermined states: $A = 0$, $B = 1$, $\texttt{ca} = p$, $q = 0$. These are **classical constants** — not quantum information.

What to do: XOR the known constant *out of* the register (zeroing it) before the peak computation, then XOR it *back in* afterward.

Example: Register `ca` holds $p = 2^{256} - 2^{32} - 977$ at termination. Before the lambda multiply: XOR p out of `ca`, returning it to $|0\rangle$. Free the register. Run the lambda multiply with the freed headroom. After the lambda multiply: XOR p back in for uncomputation.

Toffoli cost: XOR (CNOT) operations are Clifford gates — free. The teardown and recreation cost 0 Toffoli. This is a pure qubit win.

6.H. Transcript / Log Compression

The GCD must record its decisions. At each of the 530 divstep iterations, the algorithm makes data-dependent decisions: did we swap? did we subtract? These decisions are recorded in a **transcript** (also called "dialog tape") because they're needed to run the GCD backward during uncomputation. With one qubit per decision, the raw transcript is ≈ 772 bits long.

Compression techniques:

1. Base-5 codec (trailmix_ludicrous): Each 3-step window of (`subtracted`, `swap`, s_2) has 8 possible bit patterns, but only 5 are reachable. Encode each 3-step window in $\lceil \log_2(5^3) \rceil = 7$ bits instead of 9 raw bits.

```
Raw:        1 + 3*257 = 772 qubits
Compressed: 1 Step0 (2b) + 85 Triples (7b each) + 1 Pair tail (5b) + t1 flag (1b) = 603
    qubits
```

2. K5 exact 11-bit codec: Enumerate all reachable 5-step transcript windows exhaustively. If there are exactly $2^{11} = 2048$ reachable patterns, encode each in 11 bits. Information-theoretically optimal. Implemented via a borrowed-ancilla 17-CCX reversible codec. Converts 1203q→1193q.

3. Three-step tail codec (32 symbols): The last few GCD steps are heavily constrained. Enumerate the exact reachable set (32 symbols), encode in 5 bits. The 1185q→1170q drop.

4. Streaming decompression: Instead of decompressing the entire transcript before use, decompress one window at a time, use it, then immediately clear that window. Only one window is "open" at a time, reducing peak overhead from the codec.

The hard constraint: The codec must be a **bijection** — a perfect 1-to-1 map between reachable patterns and codes. Every codec must include a self-test proving bijectivity over the full reachable support.

6.I. Plateau Decomposition: Lower Every Co-Binder Together

When multiple independent phases all tie at the same peak height, you're in plateau mode. Lowering one phase alone doesn't change the global peak.

The protocol:

1. List ALL phases tied at the current peak (not just the first one). Build a "co-binder ledger."
2. Find a *local* qubit reduction for each binder independently.
3. Verify each local reduction doesn't accidentally raise a neighboring phase.
4. Combine all compatible local reductions in one package.
5. Re-measure the combined package.

Real example — the q1170 plateau (9-way co-bind, all tied):

Phase	Local fix	Effect
Solinas square	smaller square segment notch	square peak 1170→1169
apply final ripple	low-qubit final ripple variant	ripple 1170→1162
apply double/reverse halve	fold carry parking + per-step map	1170→1169
special overflow/underflow folds	fold parking + per-step map	1170→1169
non-final apply ripple	source-as-carry suffix for one lane	1170→1169

Each was independently developed, then combined. Only when all binders were locally reduced did the global peak drop to 1169.

6.J. Dynamic Headroom Clamp

Define a **circuit-wide qubit ceiling** and have every arithmetic primitive clamp its carry/chunk width to stay under it:

```
fn target_qubit_headroom(circ: &Circuit) -> usize {
    TLM_TARGET_Q - circ.active_qubits()
}

fn varchunk_adder(circ: &mut Circuit, bits: usize) -> ... {
    let k = min(scheduled_k, target_qubit_headroom(circ));
    // ... use k as the carry chunk width ...
}
```

Effect: No single call can exceed `TLM_TARGET_Q` total qubits. **Lower `TLM_TARGET_Q` by 1 → peak drops by 1**, as long as the narrower adder is still correct.

The tradeoff: Tighter ceilings force narrower carries → more carry-chunk splits → more Toffoli. The break-even exchange rate governs when to tighten.

6.K. Lazy Carry Liveness

A specific instance of live-range shortening for carry-in qubits. The carry-in qubit was being allocated before a chunk loop and held live across all N iterations, even though it was only needed at the loop boundaries.

Fix (`TLM_FOLD_CHUNK_LAZY_CIN0`):

```
Before: carry_in allocated -> live across all N chunk iterations -> used only at
    boundaries
After:  carry_in allocated inside boundary_erase() -> used -> freed inside boundary_erase
    ()
```

The carry-in qubit's liveness is reduced to just the boundary operation. Peak saved: 1 qubit per chunk at 0 Toffoli cost.

6.L. In-Place Decode: Hold Blocks Compressed

For persistent register blocks that live across the peak, encode them compressed and decode *in place* only at the single use site.

Before: Full-width raw K2-pair block (6 lanes) held live across the whole peak. **After**: 5 compressed cells + one zero lane stored at rest; decode in place only at the use site (allocate the 6 lanes transiently, use them, re-encode to 5 cells). During the rest of the circuit, only 5 cells are allocated. This is `DIALOG_GCD_K2_APPLY_INPLACE_RAW_BLOCK=1`, the 1218→1215 qubit drop.

6.M. Borrowed-Carry Fusion

Standard carry/borrow vectors allocate fresh qubits. But if an adjacent scratch register is currently known $|0\rangle$, borrow its qubits as the carry vector instead:

```
borrowed_const_fold_carries(circ, need, borrowed_carries);
// Uses borrowed[..need] as carry vector; no fresh allocation
// At exit: borrowed restored to |0>
```

Net effect: No fresh qubits allocated for the carry prefix. The peak doesn't rise.

Hard requirement: The borrowed qubits must be genuinely $|0\rangle$ at the borrow point and restored to $|0\rangle$ before the borrower reads them again as data. Getting this wrong causes silent data corruption. (`DIALOG_GCD_SPECIAL_FOLD_BORROW_CARRIES=1`.)

6.N. Passenger Ghosting via HMR

Standard passenger relocation: Free before peak; recompute after. Requires the value to be recomputable from live data (costs Toffoli).

HMR ghost (more aggressive): Measure the passenger qubit in the Hadamard (X-basis). The qubit is freed. The measurement result (a classical bit m) is recorded. Later, reconstruct the value algebraically and apply a classically-conditioned Z correction to fix the phase.

Why this works: An X-basis measurement of $|\psi\rangle$ projects it into $|+\rangle$ or $|-\rangle$. The measurement outcome m tells you which. If $m = 1$ (got $|-\rangle$), a phase (-1) was applied to the computational-basis components — the CZ correction cancels this exactly. After the correction, the measured qubit is factored out and can be freed without information loss.

In the q980 EC point addition:

- dy (256-bit) computed before the GCD inversion. After GCD, λ and dx are live.
- The identity $dy = \lambda \cdot dx$ holds by construction.
- HMR-ghost dy before the GCD peak (measure it, record m).
- At the end: reconstruct $dy = \lambda \cdot dx$, apply phase correction via m.
- Freed 256 qubits across the peak at the cost of one modular multiply for reconstruction.

6.O. EEA Register Sharing — The Low-Qubit Frontier

The deepest qubit-reduction idea, and the headline technique for the low-qubit competition (§1.D, track 2 — minimize peak qubits, Toffoli unconstrained). This is what drives the lowest qubit counts ever recorded on the challenge: the 948q → 851q → 829q descent. Because that competition does not score Toffoli at all, register sharing is free to spend gates lavishly in exchange for lanes — exactly the trade it makes.

The naive EEA register budget. The extended Euclidean algorithm carries four full-width 256-bit registers through all ∼530 steps:

- A, B — the two operands (the shrinking u, v)
- CA, CB — their two Bézout cofactors (the running a, b with $a \cdot dx + b \cdot p = \text{gcd}$)

Naively that is $4 \times 256 = 1024$ qubits just for the inversion core, before any scratch.

The sharing observation (Proos–Zalka 2003, the origin of "inversion dominates point-add space"): the operands and their cofactors are *complementary in size*. As the algorithm runs, operand A loses high bits (it shrinks toward 1) while its cofactor CA grows from zero. At every step the sum $\text{bitlen}(A) + \text{bitlen}(CA)$ is bounded by roughly one word. So A and CA can be **packed into a single physical 256-bit register**: A in the high part, CA in the low part, with a small *bit-length tracker* recording where the boundary sits. As A shrinks and CA grows, the boundary simply slides — but the total never exceeds one word.

```
Naive:   [====== A ======][== CA (mostly zero) ==]   two separate 256-bit words
         [====== B ======][== CB (mostly zero) ==]

Shared:  [== A ==|== CA ==]   ONE word; boundary slides right as A shrinks, CA grows
         [== B ==|== CB ==]   ONE word; a 9-bit length tracker remembers the split
```

Four full registers collapse to **two "work words" + a handful of small length/control trackers**. This is the structural source of Luo et al. 2026's compact exact reversible inversion at $3n + 4\lfloor \log_2 n \rfloor$ qubits (1333q for $n = 256$ as a standalone inversion). In the Shrunken-PZ track, packing $A \| CA$ and $B \| CB$ plus dirty-catalytic borrowing and gate-hosting drove the inversion core down to `REGISTER_SHARED_PAPER_INVERSION_QUBITS = 3*256 + 52 = 820` qubits, giving an **851-qubit peak** for the whole point-add — a 97-qubit drop below the prior 948q record, and the 851→829q race then squeezed out another 22.

Where the Toffoli goes (and why that's fine here). Packing is not free in gates: a shared word has a *data-dependent boundary* — "where does A end and CA begin?" depends on values computed at runtime. So every arithmetic step must first **locate and realign that boundary with a full-width comparator**, and because the circuit is reversible, that comparator is recomputed *before and after* each hosted gate:

```
Cost to realize ONE useful Toffoli on a packed word:
   toggle_gate ; ccx ; toggle_gate
   where toggle_gate ~= a full-width comparator ~= 12 x width ~= 3,075 Toffoli at width
    256
```

Across 530 divsteps this lands the 851q route at ∼464.5M average Toffoli (vs ∼54.8M for the unshared 948q route). **Under the low-qubit objective this is a non-issue** — Toffoli does not enter the score, so spending ∼8× more of it to buy 97 qubits is precisely the intended trade. (For contrast, under the $Q \times T$ *product* objective the very same packing would be ∼250× over break-even, so this technique belongs squarely to the low-qubit track, not the score track. The two objectives have genuinely different optimal constructions — see §9 for the exchange-rate math behind that statement.)

A note on rigor

The very lowest figures (851q, 829q) come from an *analysis-oracle* accounting in `register_shared_eea.rs` — it models the achievable qubit count assuming the gate-hosting is realizable, then charges the hosting Toffoli separately, rather than emitting one fully-hardened reversible op stream end to end. Treat them as the leading **lower-bound witnesses** for the pure-qubit frontier; the lowest *fully-validated* circuit witness on this track is the 948q route. Either way, the structural mechanism — operand/cofactor packing — is the real and reusable idea.

The lesson. Register sharing is *the* route below ∼948 qubits, and the Proos–Zalka / Luo-2026 mechanism is the structural source worth knowing. It is the right tool when your objective is pure qubit count. The broader point generalizes both ways: **know which competition you are playing before pricing a lever** — a move that is a clear win for the low-qubit objective (trade unlimited Toffoli for lanes) can be a clear loss for the Q×T objective, and vice versa.

7. Toffoli Reduction: The Core Loop

1. **Measure** — get the Toffoli count; find the **hot-spot**. Note emitted vs average-executed.
2. **Classify** — identify what *kind* of Toffoli these are (table below).
3. **Pick the matching move** — the kind determines the technique.
4. **Cost it** — depth-neutral? qubit cost? worth it at the exchange rate?
5. **Re-measure** — the hot-spot migrates; repeat.

Classifying Toffoli by kind:

Kind of Toffoli at the hot-spot	**The matching move**
Carry-uncompute (reverse carry chain returning scratch to $\|0\rangle$)	Vent via MBU (§8.A)
Self-uncomputing scratch (comparators built then cleanly reversed)	Conditional replay (§8.C)
Data-controlled arithmetic (`cadd`, fold-sub on live data)	Fusion / avoid materialization
Majority gadgets (3-CCX majority in carry logic)	Ancilla-free majority (§8.L)
Redundant final cleanup	Skip it (§8.N)
Full intermediate products / wide rows	Avoid materialization (§8.E)
Adjacent algebraic stages (add/negate/subtract chains)	Algebraic fusion (§8.D)

8. Toffoli Reduction Techniques

8.A. Measurement-Based Uncomputation (MBU)

The Toffoli lifecycle problem: An AND gate costs 4 T-gates (1 Toffoli) to compute. The uncomputation normally costs another 4 T-gates. Computing + uncomputing one AND = 2 Toffoli.

MBU (Jones 2013, applied to adder carry chains by Gidney 2018):

```
1. Forward: ccx(a, b, out)       -> 4 T-gates (1 Toffoli)
2. Use 'out' in downstream logic
3. Uncompute: measure 'out' in X-basis (Hadamard, then Z-basis measure)
   Result is classical bit m in {0, 1}
   - If m = 0: done, free 'out'  -> 0 Toffoli
   - If m = 1: apply CZ(a, b)    -> 0 Toffoli (CZ is a Clifford gate)
Total uncompute cost: 0 Toffoli
```

Why does this work? When you compute `out = a AND b` into a clean $|0\rangle$ ancilla, the system is in state $|a, b, a \cdot b\rangle$. Measuring `out` in the X-basis either succeeds cleanly ($m = 0$) or applies a phase kick (-1) to the $|a = 1, b = 1\rangle$ component ($m = 1$). The CZ gate corrects exactly that phase. After correction, `out` is disentangled from (a, b) and can be freed.

The key insight: **measurement collapses entanglement for free**. Coherent uncomputation must undo entanglement gate-by-gate; measurement does it in one shot.

The tradeoff: The `out` qubit must remain live from when it's computed to when it's measured (typically between the forward and reverse carry chains). This costs +1 peak qubit during that window.

Practical effect on adders: An n-bit ripple-carry adder normally costs $\approx 2n$ Toffoli (n forward, n reverse). With MBU on every carry-uncompute:

$$\text{Total adder cost} = n \text{ Toffoli (forward only)} + 0 \text{ (measured uncompute)} = n \text{ Toffoli}$$

This halves the adder cost — the most important constant-factor improvement in the SOTA.

8.B. The Vent Dial

MBU gives a precise, tunable dial. In a hybrid carry adder with k vents:

$$\begin{aligned} \text{Toffoli cost} &= 3n - 2 - k \\ \text{Peak qubits} &= \text{baseline} + k \quad \text{(held only during forward}\leftrightarrow\text{reverse window)} \end{aligned}$$

So each vent is exactly: -1 **Toffoli,** $+1$ **temporary peak qubit**.

When to vent:

- **Off-peak phases**: vent aggressively. The +1 qubit is off-peak, no score impact. Pure savings.
- **On-peak phases**: don't vent, or only if the Toffoli savings outweigh the qubit cost at the current exchange rate.

In `trailmix_ludicrous`, the schedule bakes vent budgets per call: each call's vent count is set to exactly `TLM_TARGET_Q` − `active_qubits`, spending every spare qubit below the ceiling on Toffoli savings. Calls at the ceiling get 0 vents.

Gidney 2025 streaming vented adder (open)

arXiv:2507.23079 uses 2 clean + $(n-2)$ dirty ancilla for $\approx 3n$ Toffoli. Partially implemented in `venting.rs` but **currently leaks phase** — dirty ancilla phase correction logic is incomplete. Expected $\approx n/4$ Toffoli savings per adder once fixed.

8.C. Conditional / Measured Replay (Average-Executed Only)

The idea: If a comparator flag is $|0\rangle$ on most inputs, measure the flag instead of keeping it coherent. Then replay the dependent computation conditionally on the measurement outcome.

```
1. Compute flag F into scratch ancilla (costs C Toffoli)
2. Measure F in X-basis -> classical bit m
3. If m = 0: dependent block is a no-op on this shot  -> 0 Toffoli from the block
4. If m = 1: replay the block classically conditioned on m  -> full cost
Average cost = C + Pr[F=1] * (block cost)
```

When F fires on fraction f of inputs: average-executed drops from (block cost) to $f \times$ (block cost). If $f = 0.05$, you pay only 5% on average.

The hard constraint: This ONLY works on **self-uncomputing scratch** — a comparator you compute, read under the condition, and cleanly reverse. It does NOT work on data-controlled operations or predicates reused downstream.

8.D. Algebraic Fusion: Collapse Add/Negate Chains

Adjacent modular-arithmetic stages that are algebraically equivalent to a simpler single operation can be collapsed to eliminate intermediate carry chains.

Example (`DIALOG_FUSE_C_FORM`):

```
Old:
  tx += 2*Qx    (mod add)
  tx = -tx      (mod negate)
  tx -= Qx      (mod sub)
  tx = -tx      (mod negate)

Algebraically (two negations cancel, constants combine):
  tx += 3*Qx    (one mod add)
```

Example (`DIALOG_FUSE_X_RESTORE`):

```
Old:
  tx = -tx      (mod negate)
  tx += Qx      (mod add)

Algebraically:
  tx = Qx - tx  (one operation)
```

Each removed negate/add/subtract eliminates a carry chain, a modular reduction fold, and a cleanup phase — saving significant Toffoli.

8.E. Witness-First / Deferred-Output Dataflow

Sometimes a cancellation only needs a *witness relation*, not the final output value. Computing the output early and then cleaning it is wasteful when the witness is cheaper.

The 4352cfb Toffoli cut (-10.8M Toffoli at fixed 980 qubits):

Key identity (from curve equation): `new_dy` $= \lambda \cdot$ `new_dx`. So to cancel λ, we do not need to compute `new_y` first. Instead:

1. Erase `dy` using: $dy = \lambda \cdot dx$ (subtract $\lambda \cdot dx$ from $dy \to$ zero)
2. Reuse the zeroed `dy` register as scratch
3. Compute `new_x`
4. Compute `new_dx` $= ox -$ `new_x`
5. Compute `new_dy` $= \lambda \cdot$ `new_dx` (the witness, not the output)
6. Cancel λ using `new_dy/new_dx`
7. Materialize `new_y` $=$ `new_dy` $- oy$ (only NOW compute the output)

Ablation:

```
zero-dy disabled, defer-y disabled:   110,227,695 emitted ops
defer-y materialization only:         105,573,887 emitted ops  (-4.6M)
both zero-dy + defer-y:                92,105,748 emitted ops  (-18.1M total)
```

General principle: Before materializing any output, ask: "does the next operation NEED the output, or just a function of it that's cheaper to compute directly?"

8.F. Karatsuba Modular Square (-22.4 Million Toffoli)

The largest single Toffoli reduction in the project's history.

The schoolbook problem: Computing λ^2 for a 256-bit number using schoolbook multiplication costs $O(n^2)$ multiplications.

Karatsuba: For a 256-bit number split as $\lambda = \mathrm{hi} \cdot 2^{128} + \mathrm{lo}$:

$$\lambda^2 = \mathrm{hi}^2 \cdot 2^{256} + \left[(\mathrm{lo} + \mathrm{hi})^2 - \mathrm{lo}^2 - \mathrm{hi}^2\right] \cdot 2^{128} + \mathrm{lo}^2$$

Instead of computing 4 products, compute only 3 squares: lo^2, hi^2, $(\mathrm{lo} + \mathrm{hi})^2$. Reconstruct the cross-term via additions (cheap Clifford operations).

Why -22.4M Toffoli: The squaring operation runs at EVERY GCD step (258 steps $\times$ 2 passes). Any $O(n^2) \to O(n^{1.585})$ improvement compounds massively across all those calls.

Density-neutral: The Karatsuba identity is a pure algebraic identity, correct for all integers.

Recursive Karatsuba: Tested on 128-bit sub-squares $\to$ net Toffoli LOSS (the overhead of the additions outweighs the sub-square savings at that level). Dead end.

8.G. NAF Recoding of Constants

Non-Adjacent Form (NAF) uses signed digits $\{-1, 0, +1\}$ to represent constants with fewer non-zero digits:

$$977 = 2^{10} - 2^5 - 2^4 + 1 \quad \text{(4 signed terms vs 6 binary terms)}$$

In NAF, no two adjacent digits are both non-zero, and the number of non-zero digits is minimal. For 977: 4 terms instead of 6, saving $\approx 33\%$ of the Solinas fold cost.

Density-neutral: The identity $977 = 1024 - 32 - 16 + 1$ is a mathematical fact, correct for all inputs.

8.H. Structural Dead Gate Skipping

Some CCX gates provably never fire because their control condition is structurally always zero.

Two flavors (with very different correctness profiles):

1. **Empirical dead-CCX** (REMOVED from SOTA): Sample $\approx$ 9 million inputs; find CCX gates whose target never flips; skip them. Problem: these gates might fire on inputs not in the sample. Removed in commit `6dafa07`.
2. **Structural dead-CCX** (CURRENT SOTA): Prove mathematically that a CCX gate never fires on ANY input using circuit structure alone (known-constant carries, exact-remainder conditions, call-site bit patterns). Named predicates in `trailmix_ludicrous`: `TLM_FFG_SKIP_STRUCTURAL_DEAD_*`, `TLM_COMPARE_SKIP_STRUCTURAL_DEAD_*`, etc. Density-neutral.

8.I. Classical Constant Folding

When one operand of an arithmetic operation is a **classical constant** (known at compile time), the operation can often be implemented with 0 Toffoli gates.

The mechanism: A `BitId` register holds a constant pattern as compile-time information. Operations with one classical operand:

- `qubit XOR 0` = identity (no gate)
- `qubit XOR 1` = Pauli X (free Clifford gate)
- `qubit + classical_constant mod p` = a series of CNOT-like operations — 0 Toffoli.

In trailmix_ludicrous: $Q = (Q_x, Q_y)$ is classical, so `coord_add3x(circ, x2, ox, qubits)` (*x2 += 3·*Q_x *mod p*) costs **0 Toffoli**.

8.J. Converged-Tail Gate Elision

In the last K iterations of the 530-step GCD, the algorithm has converged and the apply-phase `cswap` has a control that is deterministically 0 on all but rare inputs. Skip the cswap for those last K iterations.

Cost: Island-exact. Requires re-hunting the tail nonce after applying.

8.K. GAP_J2 Comparator Window Narrowing (−1.33M Toffoli)

The swap decision comparator at each GCD step checks whether $|u| > |v|$. A full 256-bit comparison would be expensive, but the GCD ensures that u and v typically differ somewhere in their top bits.

The schedule `GAP_J2[i]` (258-element table) sets the comparator window width per GCD step. The comparator scans only the top `GAP_J2[i]` bits. A 22-line edit to the table to shave ≈ 1 bit per step across 258 steps:

$$\text{Toffoli saved} = 1 \text{ bit} \times 516 \text{ comparator calls} \approx 1.33\text{M Toffoli}$$

Island-exact — requires nonce hunting.

8.L. Ancilla-Free Majority

Standard carry logic uses a 3-input majority gate with 3 Toffoli and an ancilla. The identity:

$$\text{maj}(a, k, c) = c \oplus ((a \oplus c) \land (k \oplus c))$$

implements the same majority with **2 Toffoli and no ancilla** (XOR = CNOT, AND = CCX). Peak-neutral, value-identical, pure count reduction wherever majority gates appear.

8.M. Exact-Adder Recovery

Counter-intuitive: Sometimes a truncated adder costs **more** Toffoli than the exact full-carry adder, because the truncation requires an expensive correction to handle the rare carry-escape cases.

The rule: If the correction overhead of a truncated adder exceeds its savings from dropping bits, revert to the exact adder. Named: `DIALOG_GCD_APPLY_FINAL_TOPCLEAN=0` → ≈ −2,597 avg-Toffoli, value-exact.

8.N. Skip Redundant Final Cleanup

A folded/structured computation sometimes makes a final "top-clean" pass redundant — the structure ensures the cleanup bits are already zero by construction. Dropping the cleanup removes its Toffoli.

Highest-risk move

If the cleanup is not actually redundant, you get silent phase/classical dirt on some inputs. Always pair with strong `cls`/`pha`/`anc` validation before enabling.

8.O. Off-Peak Loosening (The Dual Move)

When a phase falls **off** the qubit peak (another phase is now the binder), you can WIDEN that phase's arithmetic to recover Toffoli for free:

```
Phase A: was binder at 1170q -> locally reduced to 1162q
Phase B: now binder at 1170q (unchanged)
Action: widen Phase A's carry width back toward the old value
Effect: Phase A's Toffoli drops; Peak stays at 1170 (Phase B is binding)
```

The rule: Shrinking an off-peak phase is a pure-Toffoli play. Widening an off-peak phase is a free Toffoli refund. Govern both on gates alone, not qubits.

8.P. Constprop: Automated Static Gate Elimination

The constraint-propagation pass (`CONSTPROP_MAX_ITERS`) analyzes the circuit statically and eliminates:

- **Provably-constant CCX gates**: control always 0 → remove; control always 1 → replace with CNOT
- **Inverse-pair cancellation**: consecutive `CCX(a,b,c)` gates that cancel algebraically
- **Affine simplification**: sequences simplifying to an affine function, implementable with fewer gates

Setting `CONSTPROP_MAX_ITERS=256` (vs default 16) runs to a deeper fixpoint, finding more opportunities: −377k Toffoli in one version.

9. The Qubit ↔ Toffoli Exchange Rate

9.A. The fundamental arithmetic

At the current SOTA (1152q × 1,364,230 avgT = 1,571,592,960 score):

$$\text{break-even rate} = \frac{\text{avgT}}{\text{peak\_q}} = \frac{1{,}364{,}230}{1152} \approx 1{,}184 \text{ Toffoli per qubit}$$

A lever that saves 1 qubit at the cost of fewer than 1,184 Toffoli **improves the score**. A lever that costs more than 1,184 Toffoli per qubit saved **worsens the score** — even though it reduced qubits.

This rate **changes with every optimization**:

- After a large Toffoli reduction: the rate drops (qubits become more valuable). Previously-marginal qubit cuts are now worth trying.
- After a large qubit reduction: the rate rises. Previously-marginal Toffoli cuts might now dominate.

9.B. The product-race caveat

A qubit drop that clears break-even can STILL lose the product race if the Toffoli-grinding track moves faster in parallel.

Concrete example (June 2026): A 1157q clamp cost ≈ 1,127 Toffoli/qubit — below break-even at landing. But the 1159q Toffoli-grind reached 1,378,242 avgT. Score comparison:

$$1159 \times 1{,}378{,}242 = 1{,}597{,}506{,}378 < 1157 \times 1{,}380{,}890 = 1{,}598{,}290{,}330$$

The 2-qubit savings was overtaken by the parallel Toffoli-cutting track.

Lesson: Always run both tracks and compare products. Never minimize qubit count in isolation.

9.C. The shelved-lever rule

A qubit lever that **failed break-even on one Toffoli base** can become favorable on a cheaper Toffoli base.

Example: The dynamic headroom clamp (`TLM_TARGET_Q`) cost ≈ 1,514 Toffoli/qubit on the schoolbook-square base. After Karatsuba reduced the square cost, the same clamp cost only ≈ 20 Toffoli/qubit on the new base — well below break-even. It won.

Rule: After any major arithmetic win (Karatsuba, algebraic fusion, structural rewrite), re-test **all previously-shelved qubit levers** at the new exchange rate.

9.D. Different peak layers have different exchange rates

At any peak, there are typically two layers:

1. **Persistent floor**: long-lived state held across the whole peak (transcript, passenger values, GCD state). Often **cheap** to reduce — a denser codec or transcript streaming costs few gates to save many qubits.
2. **Transient working registers**: the current step's carry bits, arithmetic scratch. Often **expensive** to reduce — freeing them requires recompute/dirty-borrow, adding many Toffoli.

Counter-intuitively: **compress the transcript before freeing the scratch**. The transcript looks "irreducible" (structured data) but is often the cheap lever. The scratch looks "wasteful" but is often the expensive lever.

10. Density-Neutral vs Island-Exact

10.A. The fundamental distinction

- **Density-neutral (value-exact)**: The circuit produces the same output on ALL inputs. The optimization is grounded in mathematical identity or circuit structure.
- **Island-exact (distribution-specific)**: The circuit is correct only on the specific 9024 test inputs. On other inputs, it may give wrong answers.

When you apply an island-exact optimization, you need to find a new **nonce** — a 48-byte seed for the test input generator — such that all 9024 generated inputs happen to be in the safe zone. Stacking multiple island-exact optimizations makes the safe zone smaller.

10.B. The 6dafa07 pivot: empirical → structural dead-CCX

Before the current SOTA, the circuit used **empirical dead-CCX elimination**: sample 9M inputs, find CCX gates that never fire, skip them. Island-exact.

The `6dafa07` commit replaced this with **structural dead-CCX skipping**: prove from circuit structure which CCX gates can never fire on *any* input. These predicates are keyed by call-site and bit-index, not sampled data. Density-neutral.

10.C. The correctness check triple: `cls/pha/anc`

Every validation run checks three error types:

- `cls`: classical mismatch (the output value is wrong on some input)
- `pha`: phase error (the output is correct but entangled with extra phase — kills interference)
- `anc`: ancilla mismatch (a qubit failed to uncompute back to $|0\rangle$)

A correct circuit shows `0/0/0`. Measurement-based uncompute fails in `pha` (which value checks miss). Truncation failures typically show in `cls`. Missed uncompute shows in `anc`.

11. Pebbling Theory

11.A. The reversible pebble game

The **pebble game** models space-time tradeoffs in reversible computation. The computation is a directed graph (nodes = values, edges = dependencies). A "pebble" = a qubit holding that value. Peak qubits = maximum pebbles simultaneously.

11.B. Bennett pebbling: classical reversibility is expensive

Bennett (1989) showed that reversibly simulating a T-step computation using S extra pebbles costs:

$$\text{Time} = \varepsilon \cdot 2^{1/\varepsilon} \cdot T^{1+\varepsilon}/S^{\varepsilon} \quad \text{(for any } \varepsilon > 0\text{)}$$

The hidden constant $c(\varepsilon) = \varepsilon \cdot 2^{1/\varepsilon}$ grows **exponentially** as $\varepsilon \to 0$.

Concrete example: Halving the GCD transcript (S halved, $\varepsilon = 1$):

$$\text{Time blowup} \approx 1 \times 2 \times 530^2/370 \approx 1{,}518 \text{ extra GCD passes}$$

At $\approx$ 2,500 Toffoli per pass: 3.8M extra Toffoli for $\approx$ 370q savings. Break-even needs: 3.8M/1,184 $\approx$ 3,200 qubits. Actual saving: 370. Not worth it.

11.C. Quantum (spooky) pebbling: exponentially better

Gidney (2019) introduced **spooky pebbling**: also allow removing a pebble by measuring it in the X-basis, recording the measurement outcome as a classical bit. When the value is needed again, recompute it and apply a classically-conditioned phase correction.

Kornerup, Sadun, Soloveichik (2021) proved tight bounds:

$$\begin{aligned} \text{Quantum:} \quad & T_{\text{quantum}} = O(T/\varepsilon) \quad \text{[polynomial constant]} \\ \text{Classical:} \quad & T_{\text{classical}} = O(2^{1/\varepsilon} \cdot T) \quad \text{[exponential constant]} \end{aligned}$$

The quantum advantage is exponential. This is the theoretical backbone of why MBU and measurement-based techniques dominate.

11.D. Parallel spooky pebbling (Kahanamoku-Meyer et al. 2025)

For a length-ℓ sequential computation chain:

$$\text{qubits needed} = 2.47 \times \log(\ell) \quad \text{at depth } 2\ell$$

For our 530-step GCD: $2.47 \times \log(530) \approx 23$ qubits. Our current GCD state machine uses ≈ 22 qubits — already near the theoretical minimum.

12. The SOTA Circuit: How trailmix_ludicrous Works

12.A. The decisive architectural choice: Q is classical

The second point $Q = (Q_x, Q_y)$ is kept as classical `BitId` values, not quantum registers. Materialized into a transient quantum temp *only* at off-peak coordinate steps, then freed. This eliminates 512 qubits from the peak.

12.B. Two GCD passes sharing one tape

The affine point addition needs:

1. $\lambda = dy \cdot dx^{-1} \bmod p$ — inverse GCD (drive $(p, dx) \to (1, 0)$)
2. new_y_component $= \lambda \cdot (P_x - P'_x) \bmod p$ — forward GCD

Both share **one** modular-inverse engine and **one** dialog tape (transcript). This avoids having two separate inversion circuits, saving both qubits (one shared tape) and Toffoli (shared codec overhead).

12.C. The base-5 codec: 772 → 603 qubits live

At each of 257 divstep steps, the algorithm records (`subtracted`, `swap`, s_2) — 3 bits raw. But only 5 of 8 possible 3-bit patterns are reachable. Triple windows have $5^3 = 125$ reachable states, encoded in 7 bits instead of 9:

```
Layout: 1 Step0 (2b) + 85 Triples (7b each) + 1 Pair tail (5b) + 1 t1 flag (1b) = 603
    qubits
```

12.D. The vented-adder zoo

Every adder in `trailmix_ludicrous` vents its carry-uncompute via MBU:

```
hmr(carry, bit);        // X-basis measurement, 0 Toffoli
cz_if_bit(a, b, bit);   // phase correction conditioned on measurement
```

The adder family dispatches per-call by baked schedule tables. Each call's vent budget: exactly `TLM_TARGET_Q` − `active_qubits`, spending every spare qubit below the ceiling on Toffoli savings.

12.E. The deliberate island-exact components

trailmix_ludicrous includes calibrated island-exact approximations:

- **Low-54-bit +f fold** (`LSBS = 54`): reduction folds touch only low 54 bits; carry beyond bit 53 is dropped with probability $\approx 2^{-21}$ per fire.
- **Narrow-top-window comparators** (`MSBS = PAD = 21`): swap predicates scan only the top 21 bits. Mis-decision probability $\approx 2^{-21}$ per call.

12.F. Where 1152q comes from

At the SOTA (`d44cad3`):

- **603q**: compressed base-5 GCD transcript (live during both GCD passes)
- **512q**: two 256-bit coordinate registers (quantum P_x, P_y)
- $\approx$ **37q**: transient carry/vent ancilla at the peak-binding fold operations

13. The 1211→1152 Historical Journey

Transition	Technique	What was released
1211→1193 (−18q)	Live-range hole (§6.A)	GCD-apply scratch block freed during shift sub-phase; reacquired after
1193→1192 (−1q)	Truncation (§6.D)	Square segment one notch tighter
1192→1185 (−7q)	Transcript codec (§6.H)	Per-step fold carry scheduling + exact 11-bit head codec
1185→1170 (−15q)	Plateau decomp (§6.I) + 5 techniques	Tail codec, streaming apply, square rebalance, 5 co-binders
1170→1169	MBU vent on fold carry (§8.A/8.B)	`hmr+cz_if` on the $-f$ fold peak owner: 0 Toffoli to vent it
1167→1166	Step-0 swap_flag elimination	`swap_flag` becomes `Option<QubitId>`, lazy-alloc; freed one tape lane
1166→1165	Odd-u0 parking (§6.A)	`u[0]` is provably 1; park+loan the lane, restore in 2 gates
1164→1163	Source-as-carry suffix	Paid drop: surrendered apply-phase vents to free one carry lane
Toffoli: −22.4M	Karatsuba square (§8.F)	3 sub-squares instead of 4
Toffoli: −1.33M	GAP_J2 narrowing (§8.K)	22-line per-step comparator table edit
Toffoli: −377k	Constprop deeper (§8.P)	`CONSTPROP_MAX_ITERS` 16 → 256
1153→1152	Free-and-recompute cy0 (§6.E)	`cy0 = ctrl & !a0`; loan lane for 40 binding folds
Structural	6dafa07 pivot (§8.H/§10)	Replaced empirical dead-CCX with $\approx$ 15 structural predicates

Key meta-lessons:

1. **Every "floor" fell to a better encoding or a better question.** Not always a new algorithm — often a denser codec or the question "whose value is a cheap function of live qubits?"
2. **The shelved-lever rule is real.** The 1157q clamp tried early and lost; won after Karatsuba.

3. **Plateau mode is the endgame.** The final qubit drops each required coordinated packages of 4–9 independent local reductions applied simultaneously.
4. **Structural correctness wins over island-exact tricks.** The 6dafa07 pivot to structural dead-CCX eliminated the empirical approach entirely — correct for all inputs, not just the 9024.

13.A. The low-qubit competition track (a separate objective — §1.D)

Running *alongside* the Q×T product-SOTA journey above is the separate **low-qubit competition** line — the **Shrunken-PZ / Proos–Zalka divstep** family — whose objective is to minimize peak qubits with Toffoli unconstrained. This is the frontier where EEA register sharing (§6.O) shines:

Qubit record	Technique	Notes
1050q	trailmix shrunken-PZ embedded op stream	early low-qubit route
1019q	source-level dynamic-width PZ port	
988→956q	dirty-borrow / gate-hosting lever stack	
952→948q	further lane borrowing + thin schedule	lowest *fully-validated* witness
851q (`e7dd3de`, 6/23)	**EEA register sharing (§6.O), Luo 2026**	**−97q; the breakthrough**
829q (`1dd61ca`, 6/24)	more borrowing/relocation on a frozen envelope	current low-qubit frontier

These are records on the **pure-qubit objective**, where the ∼400–500M Toffoli they spend simply does not enter the scoring. The 851q and 829q figures come from an analysis-oracle accounting (§6.O) — read them as the leading **lower-bound witnesses** for the qubit floor; the 948q route is the lowest fully-validated circuit on the track. This is a *different game* from the Q×T product SOTA (1152q × 1.36M, value-exact): more qubits but ∼340× fewer Toffoli. The two minima are reached by different constructions, and which one is "best" depends entirely on which competition you are entering — see §6.O and §9.

14. Open Frontiers

14.A. Density-neutral Toffoli cuts not yet applied

1. **Gidney 2025 streaming vented adder** (arXiv:2507.23079): Uses 2 clean + $(n-2)$ dirty ancilla for $\approx 3n$ Toffoli. Partially implemented in `venting.rs` but **leaks phase** — dirty ancilla phase corrections are incomplete. Expected $\approx n/4$ Toffoli savings per adder once fixed.
2. **Apply-swap structural dead-CCX**: The GCD apply-step `cswap` block ($\approx$ 256 cswap per iter × 258 iters × 2 passes) has NO per-bit structural-dead skip. Porting the structural-dead analysis to these operations is a high-priority open lever.
3. **Extended Cuccaro structural dead-carry**: Currently checked for call indices 13–37 only. Extending to ALL Cuccaro adder call sites would save proportionally more.
4. **Apply-swap involutory pair cancellation**: The forward GCD-swap and the immedi-

ately following GCD-cswap on the same (u/v) lanes may cancel algebraically (swap twice = identity). Not yet checked structurally.

14.B. Density-neutral qubit cuts not yet applied

1. **The co-bind plateau at 1152q**: Multiple independent phases all tie at 1152q. Each needs a local reduction before the global peak drops.
2. **FFG cy0-style recompute on other binders**: The cy0 trick (§6.E) worked because `cy0` was a cheap function of live qubits. Other binder qubits may have similar cheap-function properties not yet identified.
3. **SumHiLo/shifted vents** (`TLM_SQUARE_SUMHILO_VENT`, `TLM_SQUARE_VENT_SHIFTED`): Exist in the codebase as default-OFF options. Off-peak phases could enable these for free Toffoli savings.

15. Quick-Reference Summary Tables

15.A. Qubit reduction techniques

Technique	Qubits saved	Toffoli cost	Density-neutral
Live-range hole (§6.A)	k qubits	$+2 \times C_V$	Yes
Passenger relocation (§6.B)	k qubits	0	Yes
Range-bound guard removal (§6.C)	1 per register	0	Yes
Truncation (§6.D)	varies	0	Partial
Free-and-recompute cheap value (§6.E)	1	≈ 0 (CNOTs)	Yes
Dynamic-width registers (§6.F)	varies	small	Partial
Known-constant teardown (§6.G)	k	0 (CNOT = Clifford)	Yes
Transcript compression (§6.H)	varies	decode overhead	Yes
Plateau decomposition (§6.I)	1 (global)	combined cost	Yes
Dynamic headroom clamp (§6.J)	1 per ceiling drop	small	Yes
Lazy carry liveness (§6.K)	1/chunk	0	Yes
In-place decode (§6.L)	varies	codec overhead	Yes
Borrowed-carry fusion (§6.M)	k	0	Yes
HMR ghosting (§6.N)	256	+1 modular multiply	Yes
EEA register sharing (§6.O)	~97–119 (to 851→829q)	high (irrelevant to low-q objective)	Yes — low-qubit competition

15.B. Toffoli reduction techniques

Technique	Toffoli saved	Qubit cost	Density-neutral
MBU / carry vent (§8.A/B)	1 per vent	+1 temporary	Yes
Conditional replay (§8.C)	$(1 - f) \times$ block	0	Yes
Algebraic fusion (§8.D)	varies	0	Yes
Witness-first dataflow (§8.E)	$\approx$ 13M / instance	0	Yes
Karatsuba square (§8.F)	$\approx$ 22M (one change)	0	Yes
NAF recoding (§8.G)	$\approx$ 5–10% of fold cost	0	Yes
Structural dead-gate skip (§8.H)	varies	0	Yes
Classical constant ops (§8.I)	significant	0	Yes
Converged-tail elision (§8.J)	hundreds/iter $\times K$	0	No
GAP_J2 narrowing (§8.K)	$\approx$ 1.33M / 22-line edit	0	No
Ancilla-free majority (§8.L)	1 per majority	0	Yes
Exact-adder recovery (§8.M)	$\approx$ 2,600/call	0	Yes
Redundant cleanup skip (§8.N)	varies	0	Risky
Off-peak loosening (§8.O)	free refund	0	Yes
Constprop deeper (§8.P)	$\approx$ 377k	0	Yes

15.C. Key theoretical results

Result	Source	Insight
MBU: uncompute AND for 0 T-gates	Jones 2013 (Phys Rev A); Gidney 2018 (arXiv:1709.06648)	X-measurement + CZ = free uncompute
1 Toffoli = exactly 7 T-gates	Gosset et al. 2013 (arXiv:1308.4134)	Tight lower bound
Bennett classical pebbling	Bennett 1989 (SIAM J. Comput. 18:4)	Classical: exponential constant in space savings
Quantum spooky pebbling	Kornerup, Sadun, Soloveichik 2021 (arXiv:2110.08973)	Quantum: polynomial constant (exponential advantage)
Parallel spooky pebbling	Kahanamoku-Meyer et al. 2025 (arXiv:2510.08432)	$2.47 \times \log(\ell)$ qubits for ℓ-step chain
Conditionally-clean ancilla	Khattar & Gidney 2024 (arXiv:2407.17966)	$3n$ Toffoli MCX with $\log^* n$ clean ancilla
secp256k1 ECDLP frontier	Babbush, Gidney et al. 2026 (arXiv:2603.28846)	$\leq$ 1200q / $\leq$ 90M Toffoli for 256-bit ECDLP
Karatsuba algorithm	Karatsuba & Ofman 1962	$O(n^{1.585})$ vs $O(n^2)$ for multiplication
EEA register sharing (origin)	Proos & Zalka 2003 (arXiv:quant-ph/0301141)	Inversion dominates point-add space; operand+cofactor packing
Compact exact reversible inversion	Luo et al. 2026 (arXiv:2604.02311)	$3n + 4\lfloor \log_2 n \rfloor$ qubits (1333q, n=256) via bit-length-tracked sharing

---

See also (all under `skills/ecdsafail_skills/ecdsafail-circuit-optimization/references/`):

- `density_neutral_tradeoffs.md` — detailed technique catalog with exchange rate math
- `gidney-techniques.md` — Gidney's full lever catalog with blog/paper pointers
- `external-literature-2000-2026.md` — broader academic literature map
- `REPORT_1168_wall_revamp.md` — the trailmix_ludicrous introduction
- `frontier-1211-to-1170.md` — detailed 1211→1170 step-by-step record